

Angular Moderno

16 - HttpClient no Angular Moderno

Angular Moderno

16 - HttpClient e Integração com API

Nesta aula vamos transformar o projeto em uma aplicação real conectada a uma API externa. Até agora trabalhamos com dados fixos. Agora vamos buscar dados reais via HTTP usando o **HttpClient moderno do Angular**.

Configurar HttpClient

No Angular moderno com providers funcionais

Primeira Requisição GET

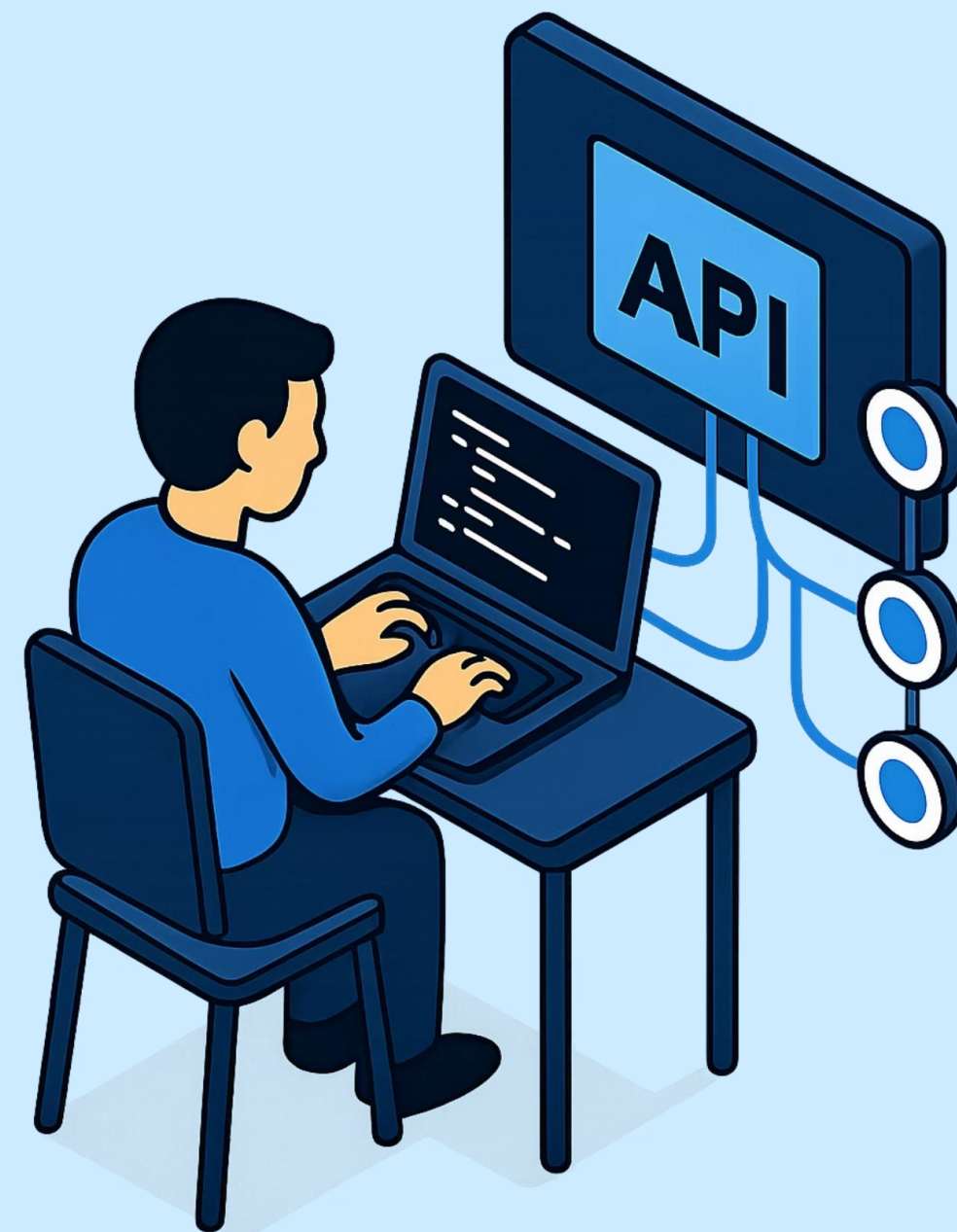
Buscar dados reais de uma API externa

Integrar com Signals

Conectar resposta da API com reatividade

Controlar Loading

Gerenciar estado de carregamento da UI



Contexto do Projeto

Antes de avançar, veja o que já construímos até aqui e qual é o próximo desafio a superar.

O que já temos

- Rotas funcionando
- Lazy Loading com `loadComponent`
- Guard funcional
- Estrutura por features

Problema Atual

Os produtos ainda estão **hardcoded** dentro do componente. Não representam dados reais e não podem ser atualizados dinamicamente.

Objetivo Agora

Substituir dados fixos por dados vindos de uma **API real** via HTTP.



Problema Atual. Didático e Intencional

O componente `lista-produtos` hoje possui limitações claras que precisamos resolver para evoluir a aplicação.

Dados Fixos

O componente contém dados hardcoded diretamente no código-fonte.

Sem Comunicação

Não há nenhuma comunicação com servidor ou fonte externa de dados.

Não é Real

Não representa um sistema real. Dados nunca mudam sem alterar o código.

⚠ Limitação crítica: não conseguimos atualizar dados dinamicamente. Vamos sair de "dados mockados" para "dados reais via HTTP".

Passo 1 — Configurar HttpClient (Angular Moderno)

SRC/APP/APP.CONFIG.TS

Adicione o provider do HttpClient no arquivo de configuração da aplicação:

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient, withFetch } from '@angular/common/http';

import { routes } from './app.routes';
import { provideClientHydration, withEventReplay } from '@angular/platform-browser';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideClientHydration(withEventReplay()),
    provideHttpClient(withFetch())
  ]
};
```

provideHttpClient

Habilita requisições HTTP em toda a aplicação

withFetch()

Usa a Fetch API moderna, evitando APIs legadas do browser

HttpClientModule

Deprecated para contexto standalone. Não usar

Por que usamos `withFetch()`?



Padrão Moderno da Web

Usa a Fetch API nativa do browser, o padrão atual para requisições HTTP.



Melhor Compatibilidade SSR

Funciona perfeitamente com Server-Side Rendering no Angular Universal.



Menor Overhead

Menos camadas de abstração, resultando em código mais leve e eficiente.

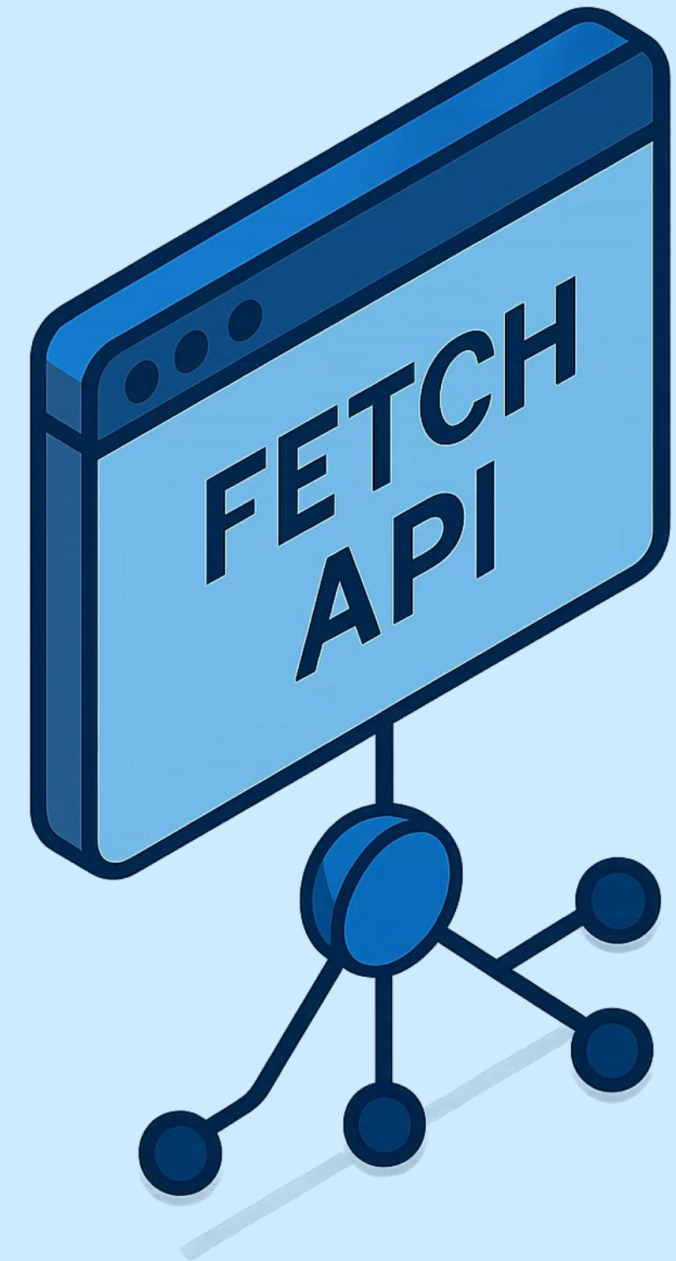


Alinhado ao Angular Moderno

Segue a filosofia de providers funcionais, nunca módulos antigos.



Regra do curso: Sempre usar providers funcionais. Nunca módulos antigos.



API que Será Utilizada

Endpoint

`https://fakestoreapi.com/products`

Retorno

Lista de produtos em formato JSON

Campos Relevantes

- `title` → nome do produto
- `price` → preço do produto

Exemplo de Resposta da API

```
[
  {
    "id": 1,
    "title": "Fjallraven Backpack",
    "price": 109.95,
    "category": "men's clothing",
    ...
  },
  {
    "id": 2,
    "title": "Mens Casual T-Shirt",
    "price": 22.30,
    ...
  }
]
```

- ❏ Vamos adaptar os dados da API para o modelo do projeto, mapeando `title` → `nome` e `price` → `preco`.

Passo 2 — Ajustar o Estado Inicial

SRC/APP/FEATURES/PRODUTOS/LISTA-PRODUTOS/LISTA-PRODUTOS.TS

O Signal de produtos precisa começar com um array vazio, pois os dados agora virão da API:

ANTES

```
produtos = signal([  
  { nome: 'Notebook', preco: 4500 }  
]);
```

Dados fixos hardcoded diretamente no Signal.

DEPOIS

```
produtos = signal<  
  { nome: string; preco: number }[]  
>([]);
```

Array começa vazio. Tipagem explícita garante segurança.

→ Array começa vazio

Nenhum dado fixo — tudo virá da API

→ Tipagem explícita

TypeScript garante que apenas objetos com `nome` e `preco` sejam aceitos

→ Signal continua reativo

A reatividade da UI é mantida.
Apenas a fonte dos dados muda

Passo 3 — Criar Estado de Carregamento

LISTA-PRODUTOS.TS

Adicione um Signal dedicado para controlar o estado de carregamento da interface (Adicionar junto aos outros signals do componente):

```
carregando = signal(true);
```

true

Carregando: requisição em andamento, exibir indicador na UI

false

Dados carregados: esconder indicador e exibir lista de produtos

 O valor inicial é **true** porque, ao iniciar o componente, a requisição ainda não foi concluída. A UI deve mostrar o estado de carregamento imediatamente.

Passo 4 – Injetar HttpClient

1. Adicionar o Import

```
import { HttpClient } from '@angular/common/http';
```

2. Injetar no Constructor

```
constructor(private http: HttpClient) {  
  
    // carrega da API  
    this.carregarProdutos();  
  
    // effects continuam iguais  
    effect(() => {  
        console.log('Lista de produtos alterada:',  
this.produtos());  
    });  
  
    effect(() => {  
        console.log('Valor total atualizado:',  
this.valorTotal());  
    });  
}
```

Continuação do Constructor

```
effect(() => {  
    if (typeof document !== 'undefined') {  
        document.title = `(${this.totalProdutos()}) Minha Loja`;  
    }  
});  
}
```

Como funciona a Injeção de Dependência

- HttpClient é injetado **automaticamente** pelo Angular
- Não instanciamos manualmente com **new**
- O Angular gerencia o ciclo de vida do serviço
- **carregarProdutos()** é chamado logo na inicialização

Passo 5 — Criar Método de Requisição Dentro da Classe do Componente ListaProdutos

```
carregarProdutos() {  
  
    // inicia loading  
    this.carregando.set(true);  
  
    this.http.get<{ title: string; price: number }[]>  
    ('https://fakestoreapi.com/products')  
    .subscribe({  
        next: (dados) => {  
  
            // Adaptação da API para o nosso projeto  
            const produtosFormatados = dados.map(p => ({  
                nome: p.title,  
                preco: p.price  
            }));  
  
            this.produtos.set(produtosFormatados);  
            this.carregando.set(false); // finaliza loading  
        },  
  
        error: (erro) => {  
            console.error('Erro ao carregar produtos:', erro);  
            this.carregando.set(false); // evita loading infinito  
        }  
    });  
}
```

01

http.get retorna Observable

A requisição é declarada mas ainda não executada

02

subscribe executa a requisição

Somente ao subscrever a requisição é disparada

03

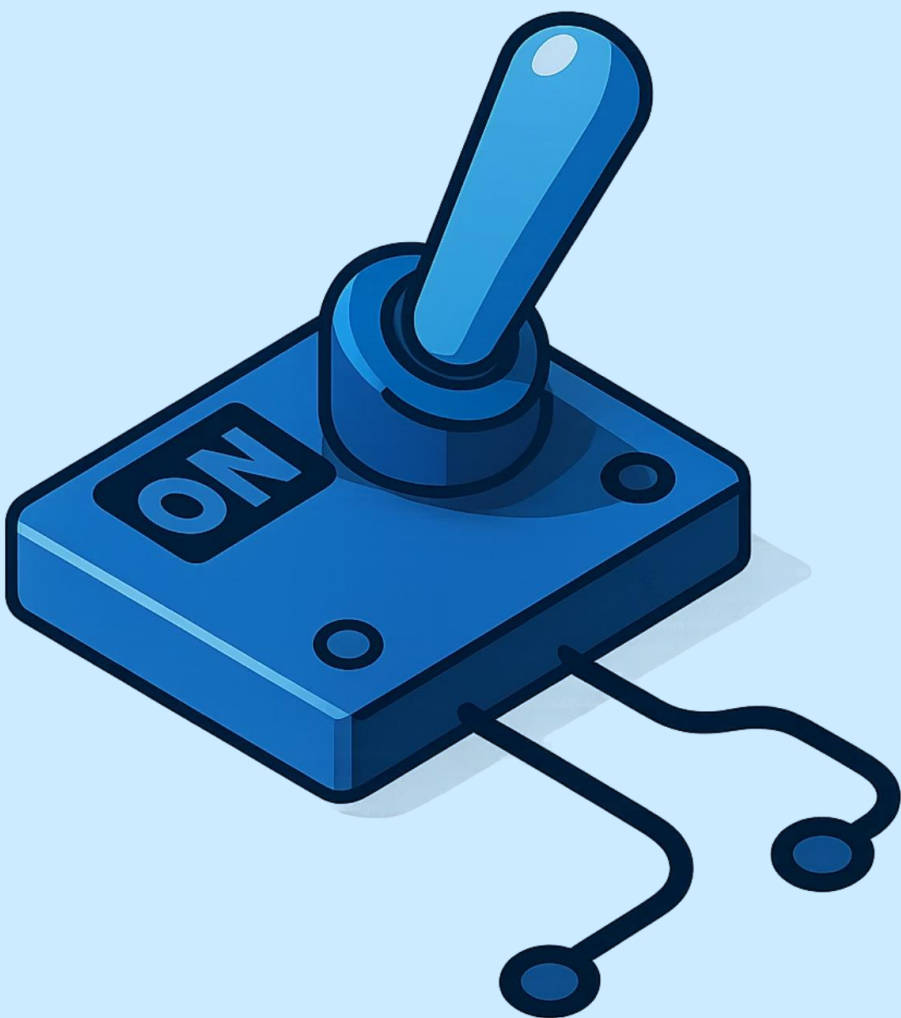
map transforma os dados

Adapta o formato da API para o modelo do projeto

04

signal.set() atualiza a UI

O Signal notifica o Angular para re-renderizar o componente



Conceito Crítico — `subscribe()`

Este é um dos conceitos mais importantes ao trabalhar com Observables no Angular. Entender o papel do `subscribe()` é fundamental.

Sem `subscribe()`

```
this.http.get('/api/produtos')  
// Nada acontece!
```

- A requisição **NÃO acontece**
- Nenhum dado é retornado
- Nenhum erro é lançado
- Código silenciosamente inativo

Com `subscribe()`

```
this.http.get('/api/produtos')  
  .subscribe({ next: ... })
```

- A requisição é **executada**
- Os dados chegam no callback `next`
- Erros são capturados em `error`
- UI é atualizada com os dados

⊗ **Regra de ouro:** Observable sem `subscribe()` é código morto. A requisição só acontece quando alguém se inscreve no Observable.

Passo 6 — Ajustar o Template

LISTA-PRODUTOS.HTML

Atualize o template para usar a sintaxe moderna do Angular e exibir o estado de carregamento:

```
<h2>Produtos:</h2>

@if (carregando()) {
  <p>Carregando produtos...</p>
}

@if (!carregando()) {
  @for (item of produtos(); track item.nome) {
    <app-produto
      [nome]="item.nome"
      [preco]="item.preco"
      (produtoSelecionado)="exibirProduto($event)"
    ></app-produto>

    <button (click)="adicionarAoCarrinho(item)">Adicionar ao Carrinho</button>
  }
}
```

@if

Substitui `*ngIf` —
sintaxe moderna
obrigatória no curso

@for

Substitui `*ngFor` — mais
performático com `track`

track \$index

Utiliza o índice como
identificador único para
cada item da lista

Integração: Observables + Signals

Entenda como as duas tecnologias se complementam para criar uma UI reativa e conectada à API.



Observable

Lida com operações **assíncronas**. Representa um fluxo de dados que pode chegar no futuro — ideal para requisições HTTP.

Signal

Atualiza a **UI automaticamente**. Quando o valor muda via `.set()`, o Angular re-renderiza apenas o que mudou.



Problema de UX — Por que o Loading Importa

Sem Loading

- Tela completamente vazia
- Usuário acha que o app travou
- Experiência frustrante
- Alta taxa de abandono

Com Loading

- Feedback visual imediato
- Usuário sabe que algo está acontecendo
- Experiência profissional
- Confiança no sistema

- ✓ O Signal **carregando** resolve exatamente este problema: controla quando mostrar o indicador e quando exibir os dados reais.

Importante — Limitação Intencional

Esta aula tem foco deliberado. Algumas práticas avançadas foram deixadas de fora intencionalmente para não sobrecarregar o aprendizado.

Service

A lógica de requisição ainda está dentro do componente. Não extraímos para um Service.

Interceptors

Não configuramos interceptors para headers, autenticação ou logging.

Tratamento Avançado de Erro

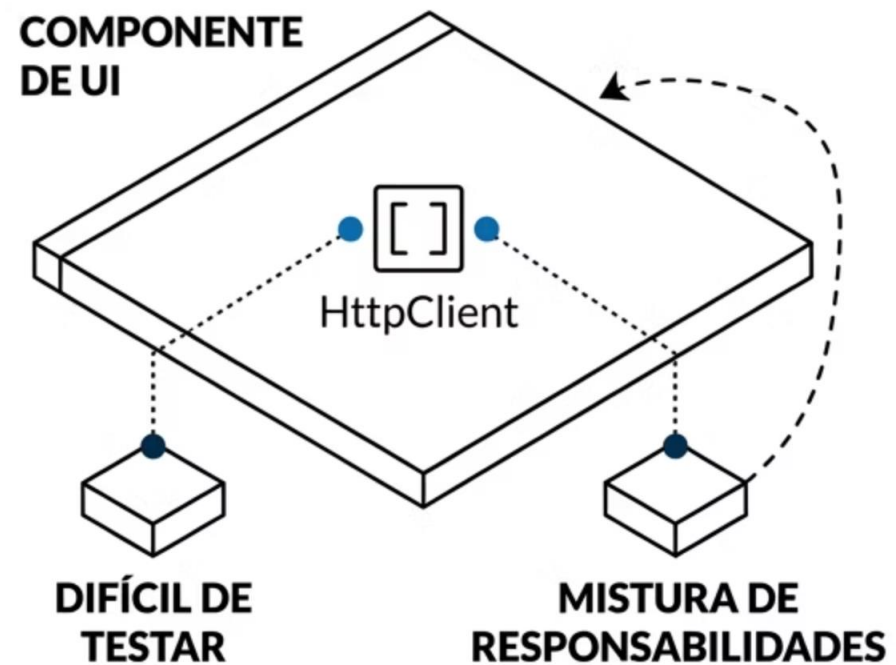
Apenas logamos o erro no console. Sem feedback visual para o usuário.

 **Motivo:** Foco total em entender o HttpClient primeiro. **Próxima aula:** Refatoração completa para Service com arquitetura profissional.

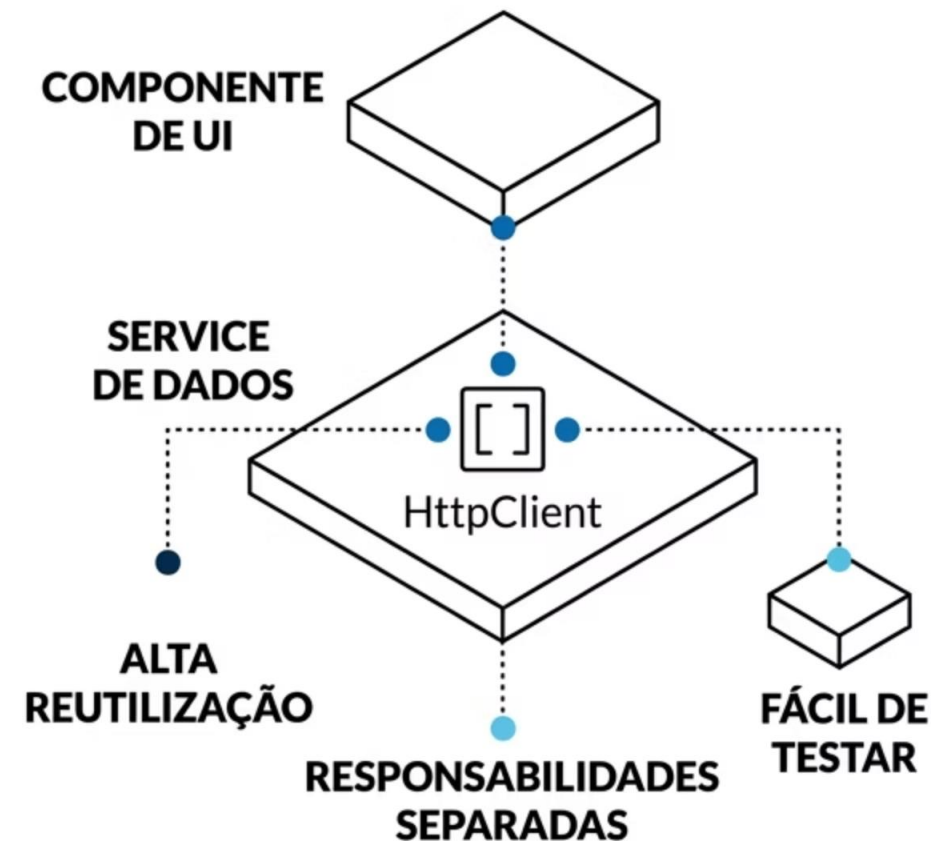
Problema Arquitetural – Intencional

Ao colocar o HttpClient diretamente no componente, criamos um problema de arquitetura que será resolvido na próxima aula.

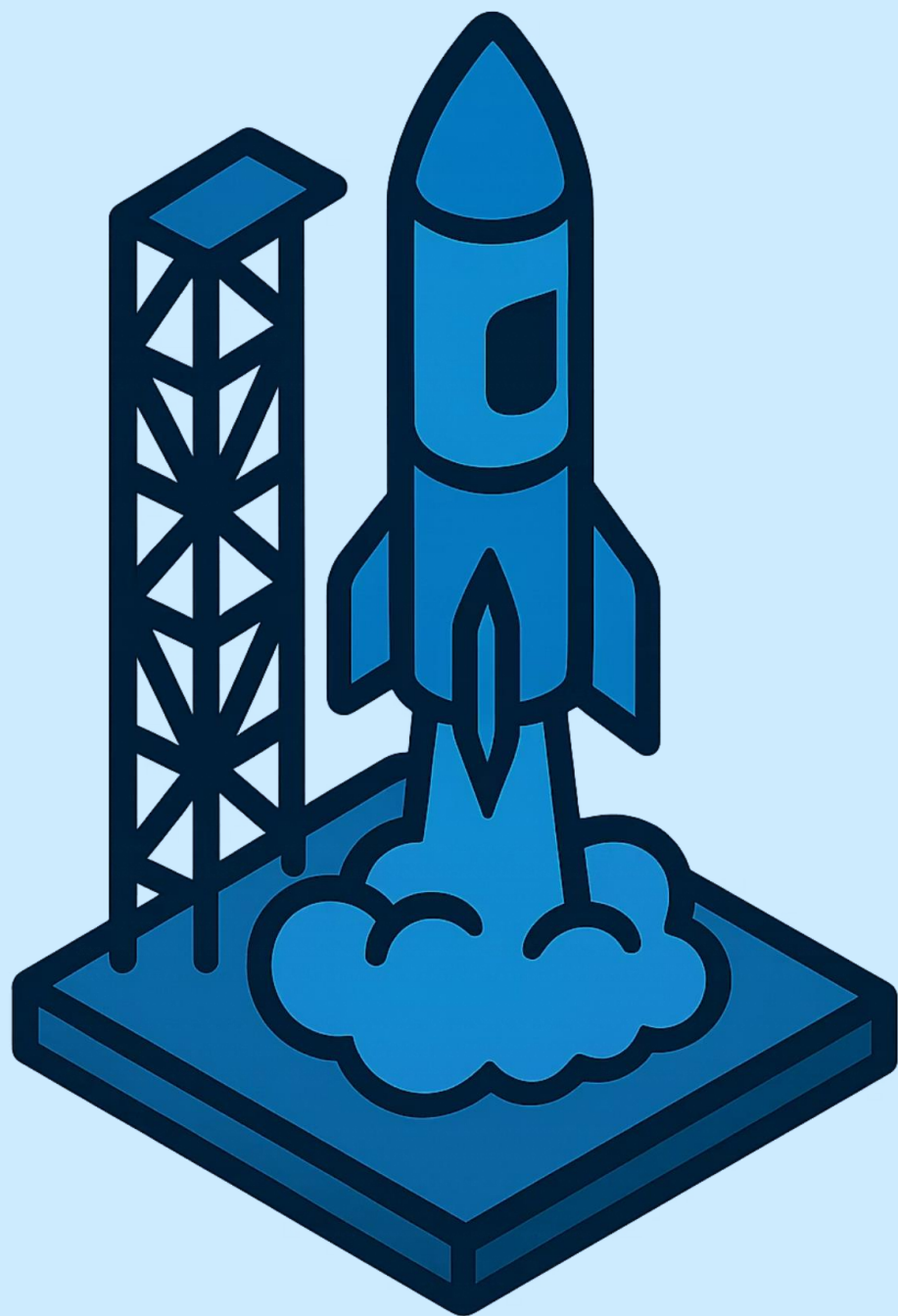
Abordagem Atual



Abordagem Futura



⚠ **Objetivo futuro:** Mover toda a lógica de requisição HTTP para um **Service** dedicado, seguindo o princípio de responsabilidade única (SRP).



Resultado Final da Aula

Parabéns! Nesta aula transformamos o projeto de um protótipo com dados fixos para uma aplicação real conectada a uma API.



API Real Integrada

Dados vindos de `fakestoreapi.com` via HTTP



Dados Dinâmicos

Nenhum dado hardcoded — tudo carregado em tempo real



UI Reativa com Signals

Interface atualiza automaticamente quando dados chegam



Loading Controlado

Feedback visual durante o carregamento dos dados

Para Alinhamento: Código completo de lista-produtos.ts

```
import { Component, signal, computed, effect } from '@angular/core';

import { HttpClient } from '@angular/common/http';

import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',

  imports: [Produto],

  templateUrl: './lista-produtos.html',

  styleUrls: ['./lista-produtos.css'],
})

export class ListaProdutos {

  // ===== SIGNALS =====

  // AGORA VEM DA API (inicia vazio)

  produtos = signal<{ nome: string; preco: number }[]>([]);

  produtoSelecioneado = signal<string | null>(null);

  // Carrinho continua igual

  carrinho = signal<{ nome: string; preco: number }[]>([]);

  // controle de carregamento

  carregando = signal(true);
```

```
// ===== COMPUTED =====

totalProdutos = computed(() => this.produtos().length);

valorTotal = computed(() => {

  return this.produtos()

    .reduce((total, item) => total + item.preco, 0);

});

quantidadeCarrinho = computed(() => this.carrinho().length);

totalCarrinho = computed(() => {

  return this.carrinho()

    .reduce((total, item) => total + item.preco, 0);

});

// ===== CONSTRUTOR =====

constructor(private http: HttpClient) {

  // carrega da API

  this.carregarProdutos();

  // effects continuam iguais

  effect(() => {

    console.log('Lista de produtos alterada:', this.produtos());

  });

  effect(() => {

    console.log('Valor total atualizado:', this.valorTotal());

  });

  effect(() => {

    if (typeof document !== 'undefined') {

      document.title = `${this.totalProdutos()} Minha Loja`;

    }

  });

}
```

```
// ===== MÉTODO HTTP (API) =====

carregarProdutos() {

  // inicia loading

  this.carregando.set(true);

  this.http.get<{ title: string; price: number }[]>

    ('https://fakestoreapi.com/products')

    .subscribe({

      next: (dados) => {

        // Adaptação da API para o nosso projeto

        const produtosFormatados = dados.map(p => ({

          nome: p.title,

          preco: p.price

        }));

        this.produtos.set(produtosFormatados);

        this.carregando.set(false); // finaliza loading

      },

      error: (erro) => {

        console.error('Erro ao carregar produtos:', erro);

        this.carregando.set(false); // evita loading infinito

      }

    });

}
```

```
// ===== MÉTODOS EXISTENTES (INALTERADOS) =====

exibirProduto(nome: string) {

  this.produtoSelecioneado.set(nome);

}

adicionarProduto() {

  this.produtos.update(listaAtual => [

    ...listaAtual,

    { nome: 'Teclado', preco: 250 }

  ]);

}

substituirProdutos() {

  this.produtos.set([

    { nome: 'Produto novo', preco: 999 }

  ]);

}

adicionarAoCarrinho(produto: { nome: string; preco: number }) {

  this.carrinho.update(listaAtual => [

    ...listaAtual,

    produto

  ]);

}
```

Dúvidas

?

